

---

## Instructions for Submission

1. File Naming and Submission:
  - a. Create a **ZIP file** containing all your solution files.
  - b. Name the ZIP file as lab12\_<ERP>.zip, where <ERP> is your ERP.
  - c. Ensure that the ZIP file contains:
    - i. C++ source files
    - ii. No unnecessary files (such as executables, temporary files, or editor configs)
  - d. Do not include TEXT(.txt) files.
2. Code Neatness:
  - a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
  - b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
  - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
  - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
  - b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
  - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
  - b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
  - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
  - a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
  - b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
  - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

---

### Lab Tasks

#### Task 1:

Create a Circle struct with:

- radius (double)

Write a function `calculateArea(Circle c)` that returns the area.

In main, create a circle with radius 5.0 and display its area.

#### Sample Output

Circle with radius 5.0 has area: 78.54

#### Task 2:

Create a Score struct with:

- subject (string)
- marks (int)

Create an array of 3 scores, read data from user, and display all scores.

#### Sample Output

Enter 3 scores:

Math: 85

Physics: 90

Chemistry: 78

Your Scores:

Math: 85

Physics: 90

Chemistry: 78

#### Task 3:

Create a Fraction struct with:

- numerator (int)
- denominator (int)

Overload the << operator to display fractions as "numerator/denominator".

#### Sample Output

Fraction: 3/4

#### Task 4:

Use the Fraction struct from Question 3.

Overload the + operator to add two fractions.

$a/b + c/d = (ad + bc)/(b*d)$

#### Sample Output

$1/2 + 1/3 = 5/6$

Lab #12

29<sup>th</sup> November, 2025

**Task 5:**

Create a Product struct with:

- name (string)
- price (double)

Write a function isCheaper(Product p1, Product p2) that returns true if p1 is cheaper.

Test with 2 products.

**Sample Output**

Laptop (\$800) is cheaper than Phone (\$1000)? No

**Task 6:**

Create a Box struct with:

- length (int)
- volume (int pointer)

In main:

1. Create a Box with length = 5
2. Allocate memory for volume
3. Store volume value (length<sup>3</sup>)
4. Display and free memory

**Sample Output**

Box length: 5

Volume: 125

**Task 7:**

Create a Counter struct with:

- count (int)

Write a function increment(Counter\* c) that increases count by 1.

Create a counter, call increment 3 times, display final count.

**Sample Output**

Initial count: 0

After 3 increments: 3

**Task 8:**

Create a Student struct with:

- name (string)
- marks (int)

1. Create an array of 3 students
2. Read student data
3. Overload << to display student info
4. Find and display student with highest marks

**Sample Output**

Enter 3 students:

...

All Students:

Alice: 85

Bob: 92

Carol: 78

Top student: Bob (92)

**Task 9 and 10 are optional but it would be best if you attempted it as it will strengthen your struct concepts!!**

### Task 9: The struct Node

Implement a `struct Node` that holds an integer value and a pointer to the next node. Overload the `<<` operator to display both the node's value and the address of the next node. In `main()`, create and play around with `Node` objects to get comfortable with them. You may try the following,

1. Connect two nodes.
2. Make a diamond of four nodes
3. A circle of N nodes pointing to a central node?

Nodes are an important concept. Make sure you're comfortable with them!

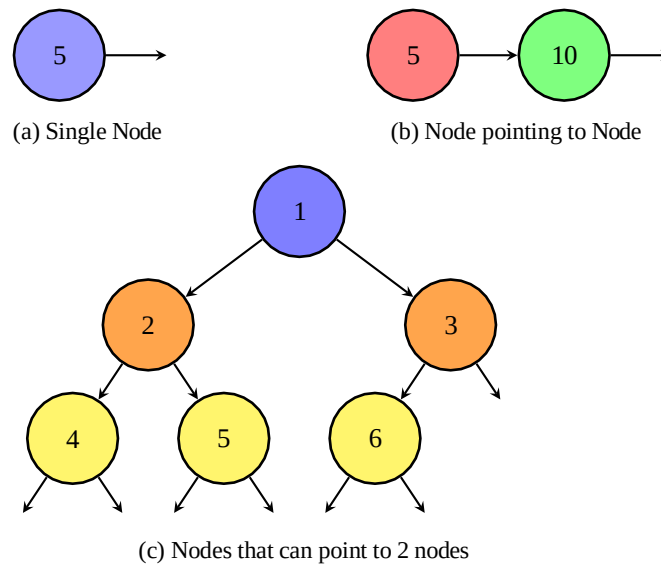


Figure 1: Understanding Nodes visually

### Task 10: Building a Node Chain

Using the `Node` structure from Task 3, implement a program that constructs a chain of nodes by appending values at the end of the node. Inside `main()` create `Node* head`, which will basically represent the start of our chain. In the beginning, when the chain is empty, `head = nullptr` (does not point to anything).

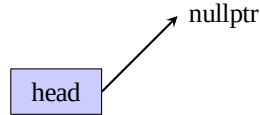


Figure 2: Initially, the chain is empty: head points to `nullptr`.

1. **Appending a new node:** Adding a new node to a chain involves 2 steps.

- (a) First we create a new node (`Node* node = new Node`) and assign `node->val = data` and `node->next = nullptr`.

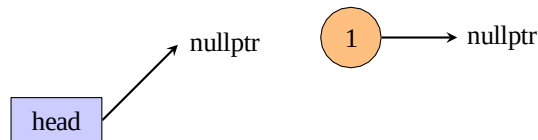


Figure 3: A new node is created with value `1` and points to `nullptr`. It is not yet added to the chain.

(b) Next we need to add this node into the end of the chain. This gives rise to 2 cases

- i. **The chain is empty:** If `head = nullptr` this indicates that the chain is empty. In this case, we simply set `head = node` and return.

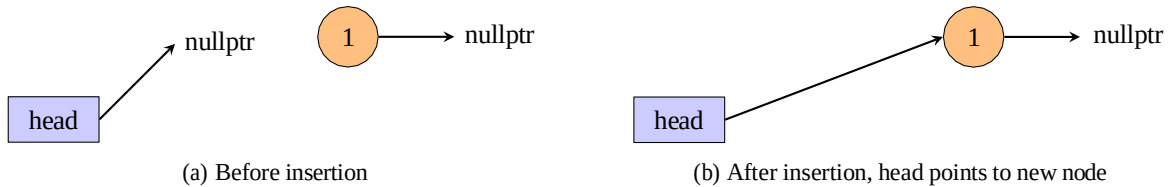


Figure 4: Step 2a: Inserting into an empty chain.

- ii. **The chain already has some elements:** In cases when `head != nullptr` we need to first find the end of the chain. Remember! `head->next` points to the next node. So by simply creating a `temp` pointer to the start of the chain (`head`) we can follow the next pointers until `temp->next = nullptr`, this indicates the end of the chain. We set `temp->next = node` and we're done.

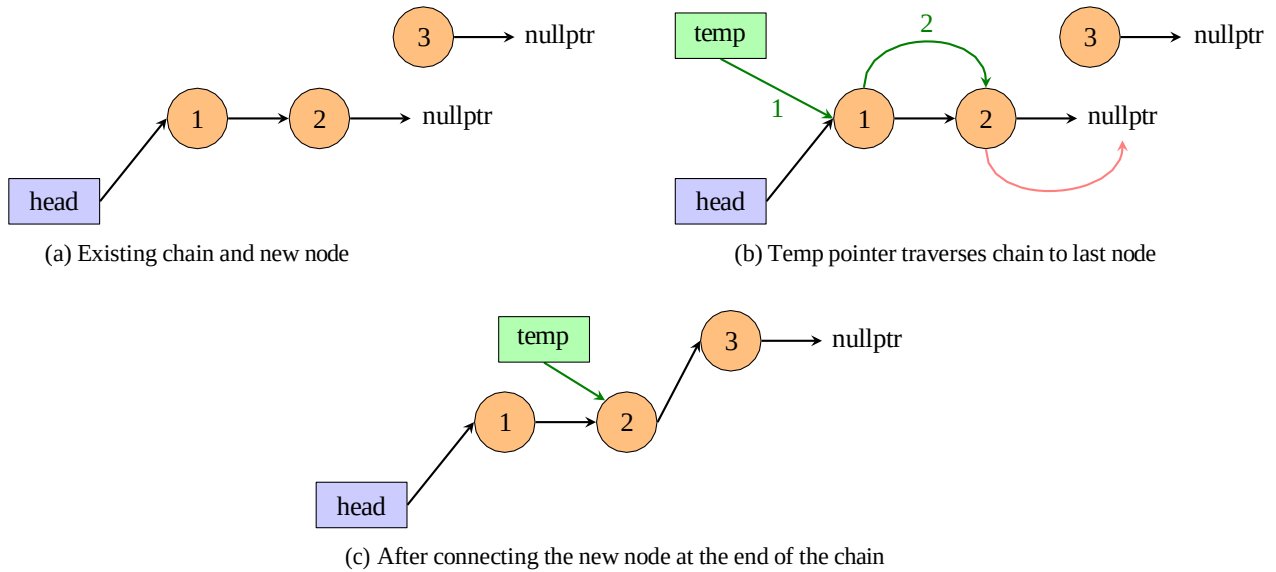


Figure 5: Appending a node to an existing chain.

Implement a function `void append(Node*& head, int val)` that can add a new node to the chain. Also, implement `void print(Node* head)` that traverses the chain starting from `head`, and prints the value of each node. **Hint:** See traversal logic for `temp` in `append()`.

The next crucial step is to make sure the allocated memory by `new` is freed via `delete`. Implement `clear(Node*& head)`, which starting from `head` deletes every node. Finally it sets `head = nullptr` to indicate that the chain is empty.

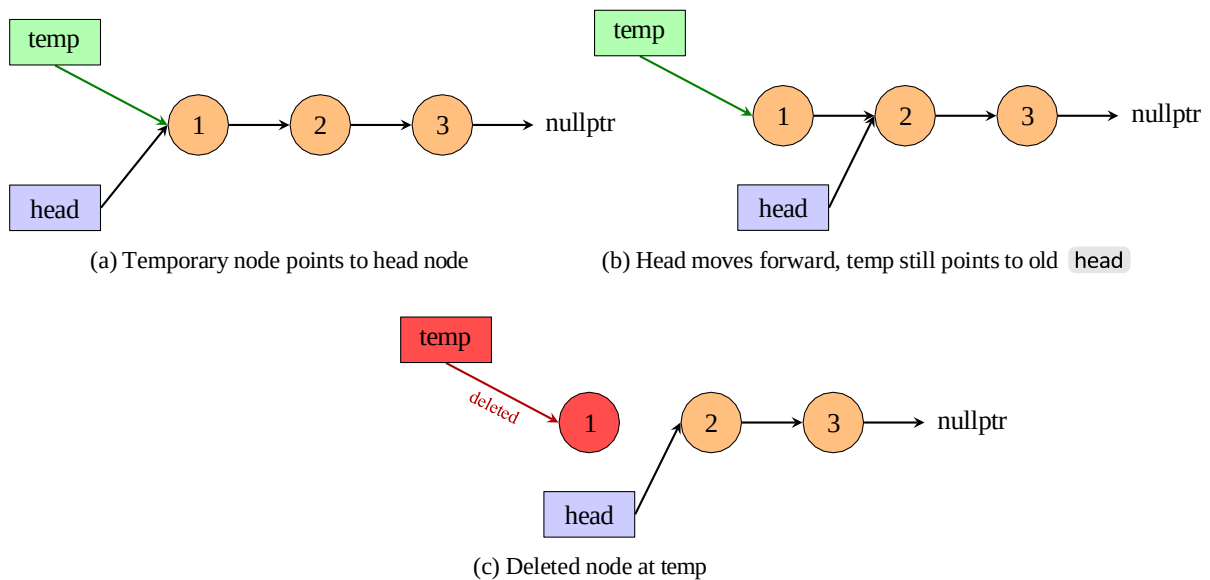


Figure 6: Deleting the first node of the chain using a temporary pointer.

Inside `main()` call `append()` to insert values. Print them using `print()` and finally call `clear()` to make sure memory is freed. Congratulations! You've just created a basic Linked List.

**Bonus:** What if we wanted to delete a node having a specific value (or at a specific position) in the chain? Draw a diagram explaining the process. You do not need to code it for now. You may use powerpoint or hand draw the diagram.